

# HLD Chapter 0: Intuitive Distributed Systems Primer & Visual Metaphors

**Core Learning Objective:** Eliminate the abstract intimidation of High-Level Design. Master every core distributed systems building block using vivid real-world analogies, understand the 10 systems side-by-side in a single matrix, and practice with the 45-minute blank interview template.

## 1. The Distributed Systems Metaphor Dictionary (Zero Jargon)

Whenever a distributed systems concept feels confusing, ground your thinking in these intuitive physical analogies:



## 2. The 13 High-Level Systems Rosetta Stone Comparison

Click on any system to jump directly into its end-to-end architecture, back-of-the-envelope math, database schema, and failure mode deep dives:

| System                                        | Traffic Type                | Read : Write Ratio   | Primary Storage               | Primary Cache                    | The Single Hardest Bottleneck                    |
|-----------------------------------------------|-----------------------------|----------------------|-------------------------------|----------------------------------|--------------------------------------------------|
| <a href="#">1. Messaging App (WhatsApp)</a>   | Bi-directional Real-Time    | 1 : 1                | <b>Cassandra</b> (LSM-Tree)   | Redis (Session registry)         | 50M open WebSockets + offline delivery           |
| <a href="#">2. Ticketing Platform</a>         | Flash Spikes (Concerts)     | 5 : 1                | <b>PostgreSQL</b> (ACID)      | Redis (Distributed lock + TTL)   | Zero double-booking across 50,000 users in 5s    |
| <a href="#">3. Photo Feed (Instagram)</a>     | Read-Heavy (Photos)         | 50 : 1               | <b>Cassandra</b> + S3         | CDN (Edge) + Redis (Timelines)   | Celebrity fan-out on write explosion             |
| <a href="#">4. Distributed Scheduler</a>      | Compute / Batch Heavy       | 1 : 1                | <b>PostgreSQL</b> + etcd      | Timing Wheel in RAM              | Worker crash failure recovery + leader election  |
| <a href="#">5. Video Streaming (YouTube)</a>  | High Bandwidth Egress       | 100 : 1              | <b>S3</b> (Object Store)      | Multi-Tier CDN Edge              | Adaptive Bitrate chunking without playback stall |
| <a href="#">6. E-Commerce (Amazon)</a>        | Financial Transactions      | 10 : 1               | <b>MySQL</b> (Partitioned)    | Redis (Atomic Lua inventory)     | Multi-service inventory/payment Saga rollback    |
| <a href="#">7. Proximity Service (Yelp)</a>   | Geospatial Read-Heavy       | 100 : 1              | <b>PostGIS</b> + ScyllaDB     | Redis (H3 Spatial Indices)       | Fast nearest-neighbor search without 2D lag      |
| <a href="#">8. Mutual Match (Tinder)</a>      | High Write / Swipes         | 1 : 1                | <b>DynamoDB</b> / Cassandra   | Redis (Sets of user likes)       | Sub-10ms mutual match detection on swipes        |
| <a href="#">9. Ride Dispatch (Uber)</a>       | Massive GPS Ingestion       | 1 : 5 (Write Heavy!) | <b>CockroachDB</b> (Trips)    | Redis (H3 Spatial driver grid)   | 1.25M GPS writes/sec without DB meltdown         |
| <a href="#">10. Social Feed (Twitter/X)</a>   | Read-Heavy Microblogging    | 10 : 1               | <b>Cassandra</b> + S3         | Redis Cluster (800 IDs timeline) | Celebrity fan-out hybrid push-pull architecture  |
| <a href="#">11. Distributed Cache (Redis)</a> | Ultra Low-Latency In-Memory | 10 : 1               | <b>Memory</b> (RAM + RDB/AOF) | Redis Master-                    | Split-brain prevention &                         |

| System                                            | Traffic Type                  | Read :<br>Write<br>Ratio     | Primary<br>Storage           | Primary<br>Cache                  | The Single Hardest<br>Bottleneck                                      |
|---------------------------------------------------|-------------------------------|------------------------------|------------------------------|-----------------------------------|-----------------------------------------------------------------------|
|                                                   |                               |                              |                              | Replica<br>Shards                 | consistent hash<br>slot rebalancing                                   |
| <a href="#">12. Search Engine (Elasticsearch)</a> | High-Volume<br>Inverted Index | 5 : 1                        | Lucene<br>Segments<br>(Disk) | File System<br>Page Cache         | Near real-time<br>index refresh &<br>segment merge<br>throttling      |
| <a href="#">13. Time-Series (Prometheus)</a>      | Append-Only<br>Metrics Flush  | 1 : 100<br>(Write<br>Heavy!) | Gorilla TSDB<br>Chunking     | Head<br>Chunk<br>Memory<br>Buffer | 10M metric<br>samples/sec<br>ingestion &<br>compactor<br>downsampling |

### 3. The 45-Minute Printable Interview Cheat Template

Print this 1-page canvas to structure your whiteboard during mock interviews:



### Further Reading

- [System Design Primer](#)
- [AWS Builders' Library](#)
- [Wikipedia: Eventual consistency](#)

### Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. What does 'eventual consistency' mean?

If no new updates occur, all replicas will converge to the same value, but reads may be stale for a while.

▼ 2. What is a single point of failure?

A component whose failure takes the whole system down; remove it with redundancy.

▼ 3. What is idempotency?

Repeating an operation gives the same result as doing it once, which makes retries safe.

▼ 4. What is backpressure?

Slowing producers when consumers cannot keep up so queues stay bounded.